

WEEK 03 MODULE
Programming Practice

TOPIC : 1 QUICK SORT

a) Easy

que. 2 **What is Sorting?**

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort



Quick Sort

Definition : Quick Sort is a Divide and Conquer sorting algorithm.

1. Choose a pivot element.
2. Rearrange the array so that:
 1. Elements smaller than the pivot go to the left.
 2. Elements greater than the pivot go to the right.
3. Repeat the same process for the left and right subarrays until the entire array is sorted.

Key Points:-

- Uses Divide and Conquer.
- Selects a pivot element.
- Partitions the array around the pivot.
- Recursively sorts the left and right subarrays.
- In-place sorting algorithm (requires very little extra memory).
- Average Time Complexity: $O(n \log n)$.
- Worst Case Time Complexity: $O(n^2)$ (when poor pivots are repeatedly chosen).

How It Works:-

38 27 43 3 9 82 10

Choose Pivot = **38**

27 3 9 10 | 38 | 43 82

↓

Sort left part → **3 9 10 27**

Sort right part → **43 82**

↓

3 9 10 27 38 43 82

Problem Statement

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement Quick Sort to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

1. Question Analysis

A cloud infrastructure company collects the average response time of servers located across different geographical locations. Since the response times are stored in the order they are received, the records are unsorted. To generate performance reports and identify high-latency servers, the operations team requires the response times to be arranged from the fastest server to the slowest server.

As the amount of data can become very large, an efficient in-place sorting algorithm is required. Therefore, the task is to implement the **Quick Sort** algorithm to sort the server response times in ascending order without using any built-in sorting methods.

2. Problem Analysis

The given problem requires sorting an unsorted list of server response times in ascending order using the Quick Sort algorithm. Since the company manages a large number of servers, the sorting technique should efficiently handle large datasets while minimizing unnecessary memory usage.

Quick Sort solves this problem by selecting a pivot element, partitioning the array such that elements smaller than the pivot are placed on its left and larger elements on its right, and then recursively applying the same process to the resulting subarrays. Once all recursive calls are completed, the response times become completely sorted in ascending order, enabling the operations team to generate accurate performance reports.

3. Algorithm Selection Analysis

Selected Algorithm: Quick Sort

- Quick Sort follows the Divide and Conquer technique.
- It partitions the array around a pivot element.
- It sorts the array in-place, requiring very little extra memory.
- It is highly efficient for sorting large datasets.
- It recursively sorts the left and right partitions until the entire array becomes sorted.
- It arranges the server response times in ascending order without using any built-in sorting methods.
- Hence, Quick Sort is the most appropriate algorithm for solving this problem.

4. Points to Remember

- Quick Sort follows the Divide and Conquer technique.
- It selects a pivot element during each partition.
- Elements smaller than the pivot are placed on the left side.
- Elements greater than the pivot are placed on the right side.
- After partitioning, the pivot reaches its correct sorted position.
- The same process is recursively applied to the left and right subarrays.
- Quick Sort performs sorting in-place, requiring very little extra memory.

- Time Complexity
 - Best Case : $O(N \log N)$
 - Average Case : $O(N \log N)$
 - Worst Case : $O(N^2)$
- Space Complexity : $O(\log N)$ (Recursive Call Stack)

5. Core Idea / Logic

- Select a pivot element from the array.
- Partition the array so that all elements smaller than the pivot are placed on its left and all greater elements are placed on its right.
- Place the pivot at its correct sorted position.
- Recursively apply the same partitioning process to the left subarray.
- Recursively apply the same partitioning process to the right subarray.
- Continue this process until every subarray contains zero or one element.
- Display the final sorted array in ascending order.

6. Execution Flow

Part A : Execution Flow

Input Phase

1. The program starts execution from the main() method.
2. The user enters the number of server response times (N), each of which is validated before being stored in the array.
3. The program validates whether N is within the specified range.
4. An integer array of size N is created.
5. The user enters all server response times.
6. The entered response times are stored in the array.
7. The quickSort() method is called to sort the array.

Processing Phase

1. The quickSort() method receives the array along with the starting and ending indices.
2. It checks whether the current subarray contains more than one element.
3. A pivot element is selected.
4. The partition() method rearranges the elements such that smaller elements are placed to the left of the pivot and larger elements are placed to the right.
5. The pivot is placed at its correct sorted position.
6. The quickSort() method is recursively called for the left subarray.
7. The quickSort() method is recursively called for the right subarray.
8. The partitioning process continues until all subarrays contain zero or one element.
9. The entire array becomes sorted in ascending order.

Output Phase

1. Control returns to the main() method.
2. The sorted array is traversed.
3. The server response times are displayed in ascending order.
4. The program terminates successfully

Part B : Detailed Steps

Step 1: Start the execution of the program.

Step 2: Read the number of server response times (N) from the user.

Step 3: Validate the value of N. If N is less than 1 or greater than 100000, display an error message and terminate the program.

Step 4: Create an integer array of size N to store the server response times.

Step 5: Read and store all the server response times in the array one by one, validate each score according to the specified constraints, and store the valid scores in the array.

Step 6: Call the quickSort(arr, 0, n - 1) method to sort the array.

Step 7: Check whether the current subarray contains more than one element (low < high).

Step 8: Call the partition() method to partition the current subarray.

Step 9: Select the last element of the current subarray as the pivot.

Step 10: Compare each element with the pivot and place all smaller or equal elements to its left while placing larger elements to its right.

Step 11: Place the pivot element at its correct sorted position and return its index.

Step 12: Recursively apply Quick Sort to the left and right subarrays formed around the pivot.

Step 13: Repeat the partitioning process until every subarray contains zero or one element.

Step 14: Display the sorted server response times in ascending order.

Step 15: Close the Scanner object and terminate the program.

7. Program Structure Analysis

i. Variable n

int n;

Purpose:

- Stores the total number of server response times entered by the user.
 - Determines the size of the array.
 - Defines the range of elements to be sorted.
-
- Constraint:

$$1 \leq n \leq 100000$$

means:

$$1 \leq n \leq 100000$$

In simple words:

- Minimum value of n can be 1.
- Maximum value of n can be 100000.

ii. Array arr[]

```
int[] arr = new int[n];
```

Purpose:

- Stores all server response times.
 - Acts as the main array on which Quick Sort is performed.
 - Contains the sorted response times after the sorting process is completed.
-
- Constraint:

$1 \leq \text{Response Time} \leq 10000$

In simple words:

- Minimum response time can be 1.
- Maximum response time can be 10000.

iii. Variables low, high and pivotIndex

```
int low, high, pivotIndex;
```

Purpose:

- low represents the starting index of the current subarray.
- high represents the ending index of the current subarray.
- pivotIndex stores the final position of the pivot after partitioning.
- These variables help divide the array into smaller subarrays during recursion.

iv. Variable pivot

```
int pivot = arr[high];
```

Purpose:

- Stores the pivot element selected for partitioning.
- Used to compare all other elements in the current subarray.
- Helps place the pivot at its correct sorted position.

v. Variables i and j

```
int i = low - 1;  
int j;
```

Purpose:

- i keeps track of the position where the next smaller element should be placed.
- j traverses the current subarray and compares each element with the pivot.
- Together, they help partition the array correctly.

vii. Input Loop

```
for(int i = 0; i < n; i++)  
{  
    arr[i] = sc.nextInt();  
}
```

Purpose:

- Accepts all server response times from the user.
- Stores them in the array.
- Ensures all input values are available before sorting begins.

vii. Recursive quickSort() Method

```
quickSort(arr, low, high);
```

Purpose:

- Recursively sorts the array using the Quick Sort algorithm.
- Calls the partition() method to position the pivot correctly.
- Recursively sorts the left and right subarrays until the entire array becomes sorted.

viii. Conditional Statement

```
if(low < high)
```

Purpose:

- Checks whether the current subarray contains more than one element.
- Prevents unnecessary recursive calls.
- Acts as the base condition for Quick Sort recursion.

ix. partition() Method

```
partition(arr, low, high);
```

Purpose:

- Rearranges the elements of the current subarray around the pivot.
- Places elements smaller than or equal to the pivot on the left.
- Places elements greater than the pivot on the right.
- Returns the final position of the pivot.

x. Partition Loop

```
for(int j = low; j < high; j++)
```

Purpose:

- Traverses the current subarray.
- Compares each element with the pivot.
- Rearranges the elements according to the partitioning condition.

xi. Comparison Statement

```
if(arr[j] <= pivot)
```

Purpose:

- Checks whether the current element is smaller than or equal to the pivot.
- If true, swaps the element to the left side of the pivot.
- Helps partition the array correctly.

xii. Swap Operation

```
int temp = arr[i];  
arr[i] = arr[j];  
arr[j] = temp;
```

Purpose:

- Exchanges the positions of two elements.
- Ensures smaller elements are placed before larger elements during partitioning.

xiii. Pivot Swap

```
int temp = arr[i + 1];  
arr[i + 1] = arr[high];  
arr[high] = temp;
```

Purpose:

- Places the pivot element at its correct sorted position.
- Divides the array into left and right subarrays for further sorting.

xiv. Output Loop

```
for(int i = 0; i < n; i++)  
{  
    System.out.print(arr[i] + " ");  
}
```

Purpose:

- Traverses the sorted array.
- Displays all server response times in ascending order.
- Produces the final sorted output.

xv. Another imp terms**1. Import Statement****Purpose:**

Imports the Scanner class from the java.util package to accept user input during program execution.

2. Class Declaration**Purpose:**

Defines the main class that contains all methods required to perform Merge Sort and execute the program.

3. main() Method

Purpose:

Acts as the entry point of the program and coordinates the overall execution.

Responsibilities:

- Creates the Scanner object.
- Reads the value of **N**.
- Creates the array.
- Accepts all package values.
- Calls the mergeSort() method.
- Prints the sorted array.
- Closes the Scanner.

8. Program

```
package week3_module;
import java.util.Scanner;
public class QuickSortEasy {

    // Function to partition the array
    static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    // Recursive Quick Sort function
    static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pivotIndex = partition(arr, low, high);
            quickSort(arr, low, pivotIndex - 1);    // Sort left subarray
            quickSort(arr, pivotIndex + 1, high);    // Sort right subarray
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of server response times to be sorted : ");
        int n = sc.nextInt();

        if (n < 1 || n > 100000) {
            System.out.println("Error: n must be between 1 and 100000.");
            sc.close();
            return;
        }

        int[] arr = new int[n];
        System.out.print("Enter the " + n + " server response times to be sorted in the order of their arrival :");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
}
```

```

        if (n < 1 || n > 100000) {
            System.out.println("Error: Response Time must be between 1 and 100000.");
            sc.close();
            return;
        }

        quickSort(arr, 0, n - 1);
        System.out.print("Sorted server response times : ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}

```

9. Program's output

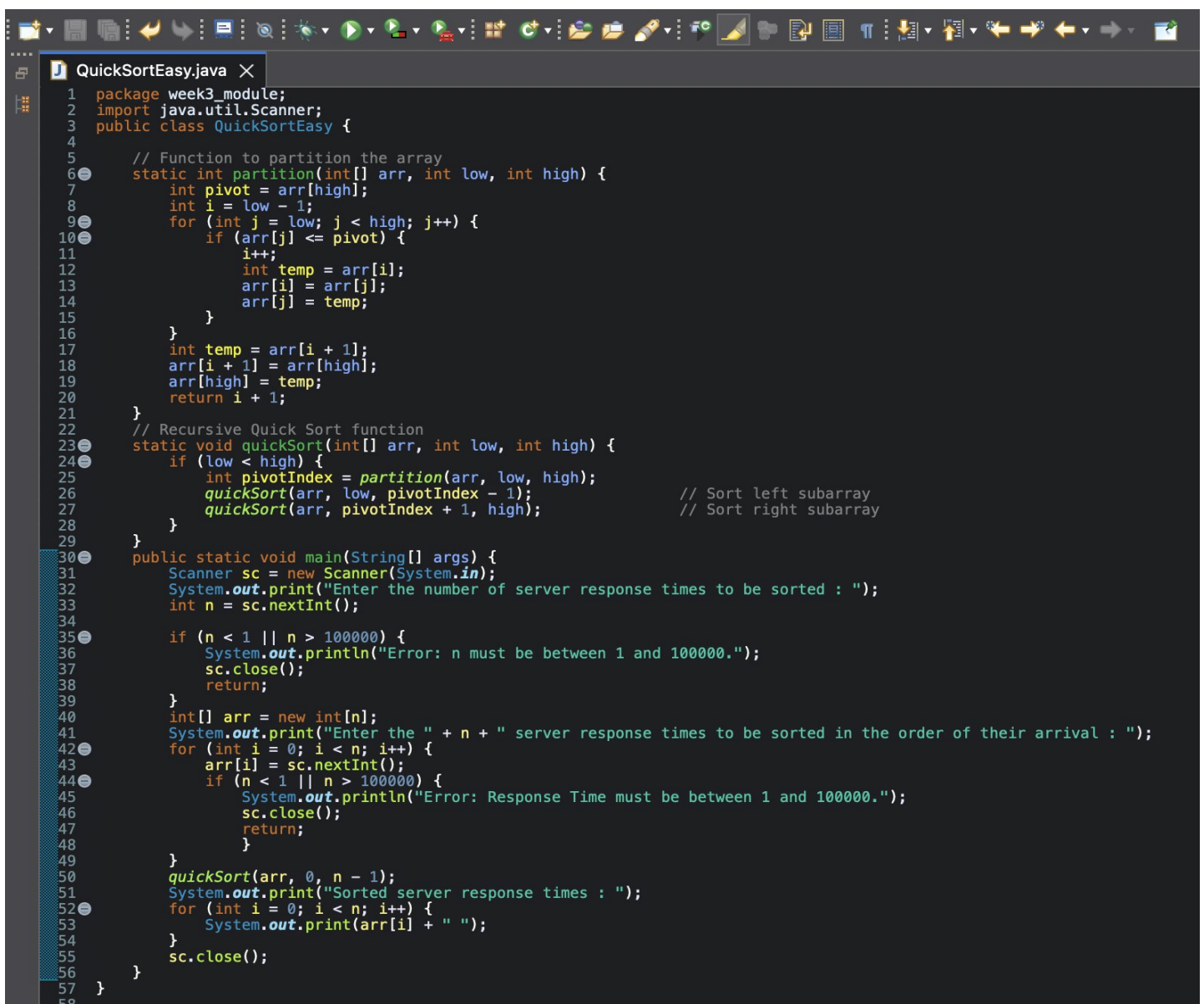
Enter the number of server response times to be sorted : 6

Enter the 6 server response times to be sorted in the order of their arrival : 120 45 78 200 60 95

Sorted server response times : 45 60 78 95 120 200

10. Screenshot

program

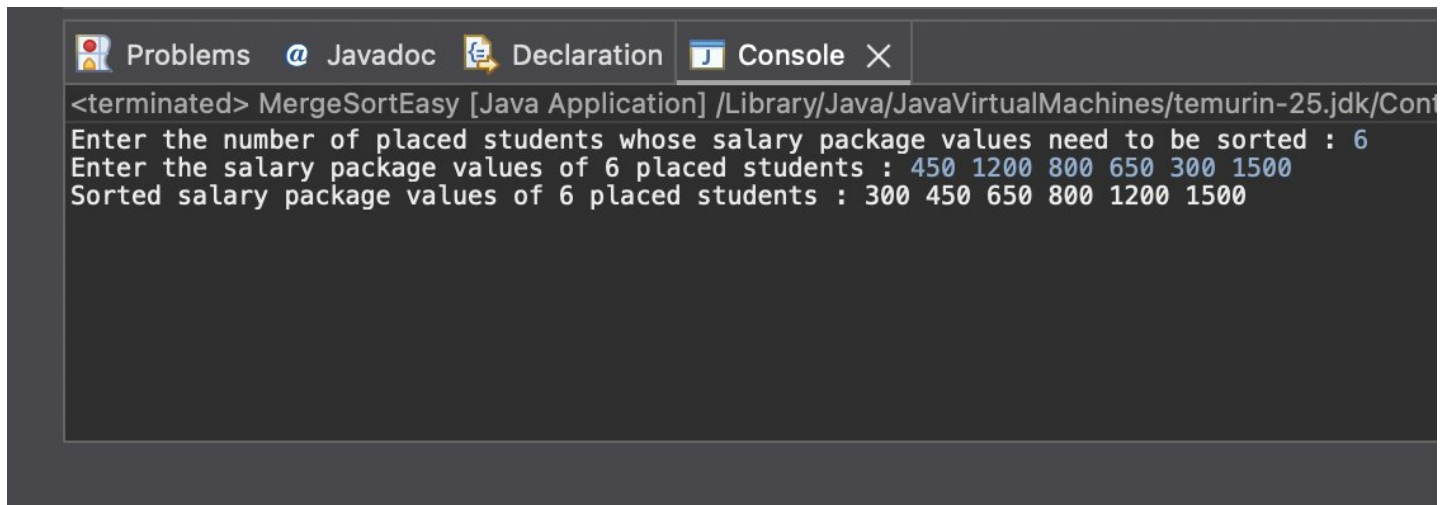


```

QuickSortEasy.java
1 package week3_module;
2 import java.util.Scanner;
3 public class QuickSortEasy {
4
5     // Function to partition the array
6     static int partition(int[] arr, int low, int high) {
7         int pivot = arr[high];
8         int i = low - 1;
9         for (int j = low; j < high; j++) {
10             if (arr[j] <= pivot) {
11                 i++;
12                 int temp = arr[i];
13                 arr[i] = arr[j];
14                 arr[j] = temp;
15             }
16         }
17         int temp = arr[i + 1];
18         arr[i + 1] = arr[high];
19         arr[high] = temp;
20         return i + 1;
21     }
22     // Recursive Quick Sort function
23     static void quickSort(int[] arr, int low, int high) {
24         if (low < high) {
25             int pivotIndex = partition(arr, low, high);
26             quickSort(arr, low, pivotIndex - 1); // Sort left subarray
27             quickSort(arr, pivotIndex + 1, high); // Sort right subarray
28         }
29     }
30     public static void main(String[] args) {
31         Scanner sc = new Scanner(System.in);
32         System.out.print("Enter the number of server response times to be sorted : ");
33         int n = sc.nextInt();
34
35         if (n < 1 || n > 100000) {
36             System.out.println("Error: n must be between 1 and 100000.");
37             sc.close();
38             return;
39         }
40         int[] arr = new int[n];
41         System.out.print("Enter the " + n + " server response times to be sorted in the order of their arrival : ");
42         for (int i = 0; i < n; i++) {
43             arr[i] = sc.nextInt();
44             if (n < 1 || n > 100000) {
45                 System.out.println("Error: Response Time must be between 1 and 100000.");
46                 sc.close();
47                 return;
48             }
49         }
50         quickSort(arr, 0, n - 1);
51         System.out.print("Sorted server response times : ");
52         for (int i = 0; i < n; i++) {
53             System.out.print(arr[i] + " ");
54         }
55         sc.close();
56     }
57 }
58

```

program's output



```
<terminated> MergeSortEasy [Java Application] /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Resources/Java/bin/java -Djava.class.path=. MergeSortEasy.java
Enter the number of placed students whose salary package values need to be sorted : 6
Enter the salary package values of 6 placed students : 450 1200 800 650 300 1500
Sorted salary package values of 6 placed students : 300 450 650 800 1200 1500
```

11. EXPLANATION :-

Program Breakdown

Every Java program starts from the main() method.

```
public static void main(String[] args)
```

Control first enters the main() method.

Step 1 : Create Scanner Object

```
Scanner sc = new Scanner(System.in);
```

Purpose

Creates a Scanner object to accept input from the user through the keyboard.

Step 2 : Read Number of Server Response Times

```
int n = sc.nextInt();
```

Suppose the input is

6

Then

$n = 6$

This means 6 server response times will be entered and sorted.

Step 3 : Validate N

```
if (n < 1 || n > 100000)
```

The problem states ; - $1 \leq N \leq 100000$

If the entered value is outside this range,

```
System.out.println("Error...");
sc.close();
return;
```

The program displays an error message and terminates.

Step 4 : Create Array

```
int[] arr = new int[n];
```

If

$n = 6$

then Java creates

```
arr = [ _ _ _ _ _ ]
```

to store the response times.

```
if(arr[i] < 1 || arr[i] > 10000)
```

Purpose:

- Checks whether the entered response time satisfies the given constraints.
- Displays an error message if an invalid response time is entered.
- Terminates the program to prevent invalid data from being processed.

Step 5 : Read Response Times

```
for(int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}
```

Suppose the input is

40 15 60 10 35 25

Then the array becomes

[40, 15, 60, 10, 35, 25]

Now all data is stored.

Step 6 : Call Quick Sort

```
quickSort(arr, 0, n - 1);
```

Since $n = 6$ the call becomes

```
quickSort(arr, 0, 5);
```

Meaning,

Sort the array from index **0** to index **5**.

Control now moves from main() to quickSort()

quickSort() Method

```
static void quickSort(int[] arr, int low, int high)
```

This method recursively sorts the array.

Step 7 : Check Base Condition

```
if(low < high)
```

If low == high

only one element remains.

Example

```
[35]
```

A single element is already sorted. So recursion stops.

Step 8 : Call partition()

```
int pivotIndex = partition(arr, low, high);
```

Instead of dividing the array at the middle like Merge Sort,

Quick Sort first calls the partition() method.

Its job is to

- choose a pivot
- place the pivot at its correct position
- rearrange the remaining elements around it

Control now moves to partition()

partition() Method

```
static int partition(int[] arr, int low, int high)
```

This is the heart of Quick Sort.

Step 9 : Select Pivot

```
int pivot = arr[high];
```

The program chooses the last element of the current subarray as the pivot.

Example :- 40 15 60 10 35 25

Pivot = 25

Step 10 : Initialize i

```
int i = low - 1;
```

Suppose low = 0 then i = -1

Why start with low - 1?

i keeps track of the last position where a smaller (or equal) element has been placed.

Initially, no smaller element has been found, so i starts one position before the first element.

As soon as a smaller element is found, i is increased and that element is moved into the correct position.

Step 11 : Traverse the Array

```
for(int j = low; j < high; j++)
```

j moves through the subarray from the first element to the element before the pivot.

Example

40 15 60 10 35 | 25

↑
Pivot

j checks

40
15
60
10
35

one by one.

Step 12 : Compare with Pivot

```
if(arr[j] <= pivot)
```

Each element is compared with the pivot.

For the example:

Pivot = 25

Comparisons:

40 > 25 ✗

15 < 25 ✓

60 > 25 ✗

10 < 25 ✓

35 > 25 ✗

Only **15** and **10** belong to the left side of the pivot.

Step 13 : Swap Elements

When a smaller element is found,

```
i++;
```

moves the boundary of the "smaller elements" section one step to the right. Then

```
int temp = arr[i];  
arr[i] = arr[j];  
arr[j] = temp;
```

swaps the current smaller element into its correct position.

After all comparisons, the array looks like

15 10 60 40 35 25

The smaller elements have been gathered on the left.

Step 14 : Place the Pivot

After the loop,

```
int temp = arr[i + 1];  
arr[i + 1] = arr[high];  
arr[high] = temp;
```

places the pivot in its correct sorted position.

The array becomes

15 10 25 40 35 60

Notice:

15 10 | 25 | 40 35 60

Everything left of 25 is smaller.

Everything right of 25 is larger.

The pivot is now permanently in its correct position.

The method returns

```
return i + 1;
```

which is the pivot's index.

Control returns to quickSort().

Step 15 : Sort Left Subarray

```
quickSort(arr, low, pivotIndex - 1);
```

Now only

15 10

is sorted.

Step 16 : Sort Right Subarray

```
quickSort(arr, pivotIndex + 1, high);
```

Now only

40 35 60

is sorted.

The same partitioning process repeats recursively.

Eventually the array becomes

10 15 25 35 40 60

Step 17 : Return to main()

Once every recursive call finishes,

control returns to the main() method.

Step 18 : Print the Sorted Array

```
for(int i = 0; i < n; i++)  
{  
    System.out.print(arr[i] + " ");  
}
```

Output

10 15 25 35 40 60

Step 19 : Close Scanner

```
sc.close();
```

The Scanner object is closed, and the program terminates successfully.

Overall Program Flow

main()



Read N



Validate N



Create Array



Read Response Times



Call quickSort()



Call partition()



Select Pivot



Partition the Array



Place Pivot at Correct Position



Recursively Sort Left Subarray and Right Subarray



Return to main()



Display Sorted Response Times



Close Scanner



End Program